AI Agents for Bioinformatics

Team members: Ozi Jawad, Carter Struck, Mack Hicks, Sona James | **Faculty adviser:** Tomasz Arodz, Ph.D. | **Partner:** VCU College of Engineering | **Mentor:** Tomasz Arodz, Ph.D

Objective

- Develop a functional multi-agent AI system using open-source large language models (LLMs) capable of processing, coding, and interpreting genomic datasets.
- Design and implement an AI-based tool that can automatically generate and execute Python code for bioinformatic tasks.
- Deliver accurate and reliable solutions generated by the AI system.
- Reduce time required to process and interpret data by 50% compared to traditional bioinformatic workflows.

Problem

- The problem is the complexity of current bioinformatic tasks dealing with large datasets.
- The goal of the agent is to simplify this process for non-bioinformaticians working on genomic datasets that often require a bioinformatician to decipher.
- To solve this problem, we propose a multi-agent AI system capable of reading genomic data, analyzing it, and producing useful and understandable insights.

Example Input Output

Prompt: **Write a Python function that calculates the GC content given a DNA sequence.**

```
def gc_content(seq):
    # START_OF_FUNCTION
    count = 0
    for i in seq:
        if i == "G" or i == "C":
            count += 1
    return count / len(seq) * 100

# End the function after implementation with marker
# END_OF_FUNCTION

# Example DNA sequences to test the function
dna1 = "ATGCGC"
dna2 = "ATATAT"
dna3 = "GGCCGGCC"

# Test the function
print(f"GC content of {dna1}: {gc_content(dna1):.2f}%")
print(f"GC content of {dna2}: {gc_content(dna2):.2f}%")
print(f"GC content of {dna3}: {gc_content(dna3):.2f}%")
```

OUTPUT:

GC content of ATGCGC: 66.67%
GC content of ATATAT: 0.00%
GC content of GGCCGGCC: 100.00%

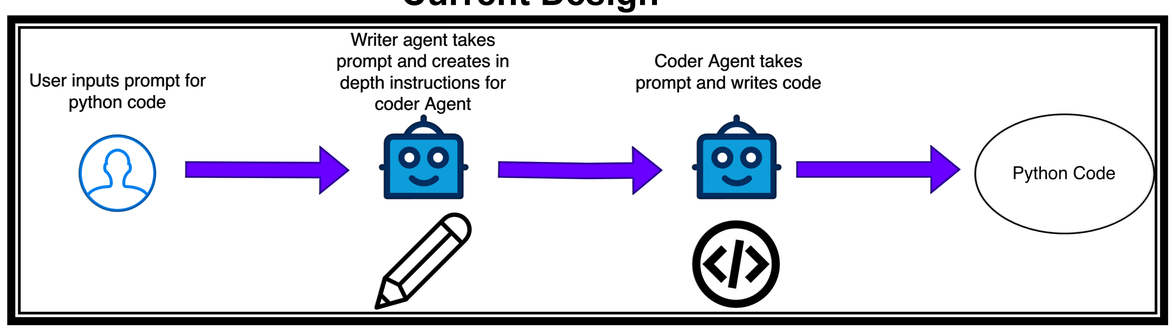
Design Constraints

- Modal Must run on NVDIA A100 40GB
- LLMs parameter sizes limited to 20 Billion
- Open Source LLMs only
- No Chinese LLMs
- Reasonable completion speed for output
- Agents must debug and produce runnable python code

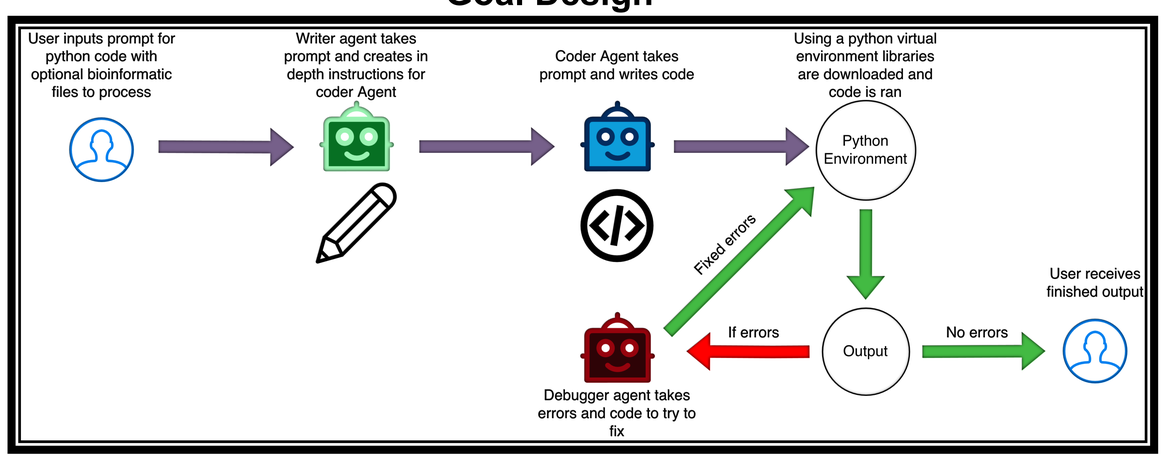
Design Choices

- We have chosen StarCoder for both the writing and the coding agents for system design purposes.
- This choice was made to ensure prototype alignment and reduce the system's complexity while designing and testing the overall workflow of the system.
- Before testing other LLMs for our specified purposes, implementing the system with a single LLM allows us to build a robust system that links the two types of LLMs needed for our project's purposes before attempting to try it on other LLMs.

Current Design



Goal Design



VCU College of Engineering